

Introduction

Every so often a LeetCode problem looks like a sliding window question on the surface but actually rewards a completely different mental model. LeetCode 1477, Find Two Non-overlapping Sub-arrays Each With Target Sum, is one of those. It gets filed under "sliding window" and "prefix sum" tags, but the real skill being tested is something interviewers care about deeply: can you take a brute-force idea that compares every pair of results, and collapse it into a single pass by carrying forward just one piece of state?

This pattern shows up constantly outside of LeetCode too — anywhere you need to combine two non-overlapping "events" optimally over a stream of data, whether that's log intervals, transaction windows, or scheduling slots. Learning to recognize it here means you'll spot it again in a system design interview or in production code that has to process something in one linear scan for performance reasons.

Problem Overview

You're given an array of positive integers and a target number. Your job is to find two subarrays — contiguous slices of the array — that don't overlap with each other, where each subarray's elements sum exactly to the target. Among all valid pairs of subarrays that satisfy this, you want the pair whose combined length (length of subarray one plus length of subarray two) is the smallest possible. If no such pair exists, return -1.

The key constraints that shape the solution: the array can hold up to 100,000 numbers, so anything quadratic is too slow. Every number is a positive integer between 1 and 1000, which matters more than it looks — because all values are positive, the running sum as you walk the array only ever increases. It never repeats a value and never decreases, which means each prefix sum is unique and lookups against it are unambiguous. The target itself can be as large as 100 million, ruling out any approach that depends on a small, fixed-size range of possible sums.

Example

Take the array `[3, 2, 2, 4, 3]` with a target of `3`.

- The element at index 0 is `3` on its own — that's a valid subarray of length 1.
- The element at index 4 is also `3` on its own — another valid subarray of length 1, and it shares no indices with the first one.

Combined length: $1 + 1 = 2$. That's the answer.

Notice there's also a subarray `[2, 2]`... no wait, $2 + 2 = 4$, not 3, so that doesn't qualify. The only valid single-element matches here are indices 0 and 4, and since they don't overlap, they're immediately the

best (and only) pair.

Intuition

Picture scanning a long receipt tape as a cashier. You don't get to scroll back and forth — you read it once, left to right, keeping a running total in your head. That mental image is exactly the shape of this problem: up to 100,000 numbers means we need something close to a single pass, not an approach that inspects every possible pair of subarrays.

The first instinct many people have is: find every subarray that sums to the target, then check every pair of them for overlap and pick the best combined length. That's the brute-force path, and it's worth understanding before optimizing away from it.

The key insight that unlocks the $O(N)$ solution is this: the sum of any contiguous stretch of the array equals one running total minus an earlier running total. If you're walking the array and keeping a cumulative sum, then a subarray ending at your current position sums to the target exactly when there was some earlier point where the running total was precisely `target` less than it is now. That turns "does a subarray summing to target exist ending here" into a single hash map lookup — no rescanning required.

Once you can detect a qualifying subarray in constant time, the second piece of intuition is what makes this genuinely $O(N)$ instead of $O(N^2)$: every time you find a new valid subarray ending at the current index, you don't need to compare it against every subarray found so far. You only need to know the single best (shortest) valid subarray that already finished before this one started. Carry that "best so far" number forward as you scan, and each new find only needs to check against that one number.

Brute Force Solution

Idea: Find all subarrays summing to the target using prefix sums (this part is unavoidable and cheap), then compare every pair of those subarrays to check for overlap and track the minimum combined length.

Algorithm: 1. Compute all subarrays whose sum equals the target — store their (start, end) index pairs. 2. For every pair of these subarrays, check if their index ranges overlap. 3. Among non-overlapping pairs, track the minimum sum of their two lengths.

Advantages: Conceptually simple, easy to reason about correctness.

Disadvantages: The number of qualifying subarrays can itself be $O(N)$ in the worst case, and comparing every pair of them is $O(N^2)$. For 100,000 elements, that's potentially 10 billion comparisons — far too slow.

Time complexity: $O(N^2)$ in the worst case. **Space complexity:** $O(N)$ to store the qualifying subarrays.

Optimal Solution

The optimal approach processes the array exactly once, maintaining three pieces of state:

State**Purpose**

<code>seen</code> (dict)	Maps a running sum $\rightarrow$ the index where that sum last occurred
<code>min_len[i]</code>	Shortest valid subarray found anywhere in <code>array[0..i]</code>
<code>best</code>	Best combined length found so far (starts at infinity)

Step-by-step:

1. Seed `seen` with `{0: -1}`. This means: "a running sum of zero occurred at index -1." It matters because a valid subarray starting at index 0 needs something to subtract against — without this seed, that subarray would be invisible to the lookup.
2. Walk the array, adding each number to a running `total`.
3. At each index, check if `total - target` exists in `seen`. If it does, the subarray from just after that earlier index up to the current index sums exactly to the target.
4. When you find such a subarray, check `min_len` at that earlier index — that tells you the best subarray that already finished before this one started. If one exists, combine the two lengths and update `best`.
5. Update the running "best single-length subarray seen so far" tracker, then record it into `min_len` at the current index.
6. Store `total` in `seen` at the current index for future lookups.
7. After the loop, return `best` if it was ever updated, otherwise return -1.

The one detail worth pausing on: you must check that the earlier index is not -1 in a way that would let a subarray pair with itself starting at index 0. A subarray beginning at index 0 has nothing valid before it, and skipping that guard would silently let the algorithm combine a stretch with a phantom zero-length partner.

Python Code

```
def min_sum_of_lengths(arr, target):
    n = len(arr)
    INF = float('inf')

    # min_len[i] = length of the shortest valid subarray ending at or before index i
    min_len = [INF] * n
    seen = {0: -1} # running sum -> index where it occurred

    total = 0
    best_so_far = INF # shortest single valid subarray seen up to current index
    best_combined = INF

    for i, num in enumerate(arr):
        total += num
        needed = total - target

        if needed in seen:
            start = seen[needed]
            current_len = i - start

            # Combine with the best subarray that finished before this one started
            if start != -1 and min_len[start] != INF:
```

```
        best_combined = min(best_combined, min_len[start] + current_len)

    best_so_far = min(best_so_far, current_len)

    min_len[i] = best_so_far
    seen[total] = i

return best_combined if best_combined != INF else -1
```

Code Walkthrough

- `min_len[i]` tracks the shortest valid subarray found anywhere from the start of the array through index `i`. It's a running "best so far" that any later subarray can look up.
- `seen` maps a running sum to the index at which it occurred. It's seeded with `{0: -1}` so subarrays starting at index 0 have something to subtract against.
- For each number, we update `total`, then compute `needed = total - target`. If `needed` is a sum we've already seen, a valid subarray exists ending here.
- `start = seen[needed]` gives us the index right before this new subarray begins. Checking `min_len[start]` tells us whether a qualifying subarray already finished by that point, and if so, we try pairing the two.
- `best_so_far` tracks the shortest single qualifying subarray ending at or before the current index — this becomes tomorrow's `min_len` entry.
- `seen[total] = i` records today's running sum for future lookups.

Dry Run

Array: `[3, 2, 2, 4, 3]`, target: `3`.

i	num	total	needed	in seen?	start	min_len[start]	best_combined	best_so_far	min_len[i]	seen update
0	3	3	0	yes (-1)	-1	skip (start == -1)	inf	1	1	{0:-1, 3:0}
1	2	5	2	no	—	—	inf	1	1	+{5:1}
2	2	7	4	no	—	—	inf	1	1	+{7:2}
3	4	11	8	no	—	—	inf	1	1	+{11:3}
4	3	14	11	yes	3	min_len[3]=1	1+1=2	1	1	+{14:4}

Final `best_combined = 2` — matching the expected answer.

Complexity Analysis

Time: O(N). Every element is processed exactly once. Dictionary insertion and lookup are O(1) average case, and array indexing is O(1).

Space: $O(N)$. The `seen` dictionary can hold up to N entries (one running sum per index, all distinct since values are positive), and `min_len` is an array of size N .

These are correct because there's no nested loop and no re-scanning — everything is computed from state carried forward from the previous iteration.

Alternative Solutions

A two-pass variant is common: compute `min_len` left-to-right in one pass, then a second array `min_len_from_right` right-to-left, and in a final pass check every valid subarray against the right-side minimum too (to catch cases where the second subarray comes before the first in your scan order, but you want the shorter one on the left conceptually). This is still $O(N)$ time and space — it's a stylistic variant of the same idea, sometimes easier to reason about because it explicitly separates "look left" and "look right" concerns. The single-pass version above is more compact but relies on the fact that non-overlapping just means "the second one starts after the first one ends," which the running index naturally enforces.

Edge Cases

- **No valid subarray exists at all** → the function correctly returns -1 since `best_combined` never updates from infinity.
- **Only one valid subarray exists** → `best_combined` stays infinity because there's never a pair, correctly returning -1.
- **Target equals a single element repeated many times** (e.g., all 1s with target 1) → many length-1 subarrays qualify; the algorithm should still find the minimum pair (length 2) without extra logic.
- **Minimum values** (array of all 1s, target 1, length 2 array) → the two shortest possible non-overlapping matches, combined length 2.
- **Maximum target with small numbers** (target larger than the sum of the whole array) → no subarray can ever sum to target, so `needed` never matches, and the function returns -1.
- **Two qualifying subarrays are adjacent** (no gap between them) → still valid, since adjacency isn't overlap; the algorithm handles this naturally because `start != -1` and the ranges don't share indices.

Common Mistakes

1. **Forgetting to seed `seen` with `{0: -1}`**. Without it, any subarray starting at index 0 is invisible to the algorithm.
2. **Not guarding against `start == -1` when combining**. This lets the algorithm silently pair a subarray with a phantom "subarray of length 0" that doesn't exist, producing wrong answers.
3. **Confusing prefix sum indices with subarray lengths**. The subarray length is `i - start`, not `i - start + 1` when `start` is treated as "one before the subarray begins" — get this off-by-one wrong and every length is skewed.

4. **Trying to solve this with a true sliding window (two pointers).** Because you need to find *pairs* of subarrays rather than a single contiguous region, a classic expand/shrink window doesn't directly apply — the prefix sum + hash map pattern is what actually fits.
5. **Updating `min_len[i]` before checking `min_len[start]` for combination.** If you update the running minimum before doing the lookup, you can accidentally use a subarray's own information to "combine with itself," corrupting the result.

Interview Questions

- How would this change if array elements could be negative?
- Can you return the actual subarrays instead of just the combined length?
- What if you needed three non-overlapping subarrays instead of two?
- How would you adapt this to a streaming input where you can't store the whole array?
- Why does the positive-integer constraint make the hash map approach reliable here?

Similar Problems

- **LeetCode 560 (Subarray Sum Equals K)** — same prefix sum + hash map foundation, but only asks for a count of qualifying subarrays, not a pairing.
- **LeetCode 325 (Maximum Size Subarray Sum Equals k)** — nearly identical prefix sum technique, but optimizes for the longest single subarray instead of shortest combined pair.
- **LeetCode 209 (Minimum Size Subarray Sum)** — a genuine sliding window problem, useful for contrasting when two pointers apply versus when prefix sums are the right tool.

Key Takeaways

The lesson here isn't really about subarrays — it's about recognizing when a brute-force "compare every pair" approach can be replaced by "carry forward the single best candidate as you scan." Prefix sums convert range-sum questions into $O(1)$ lookups, and tracking a running minimum alongside them turns a pairing problem into a single linear pass. That combination — hash map for detection, running minimum for combination — is a pattern worth having ready for any interview question that asks you to combine two non-overlapping results optimally.

Watch the Video

Watching the full walkthrough makes the index bookkeeping much easier to internalize, especially the guard against pairing with a phantom subarray. Check it out here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series dedicated to teaching real interview-relevant algorithms with a focus on the *why* behind each optimization, not just the code that passes the test cases. Every episode builds toward the same goal: making the leap from brute force to optimal feel obvious rather than magical.

Call To Action

If this walkthrough helped the pattern click, give the video a like on YouTube and subscribe for more deep dives into interview-level algorithms. For written breakdowns delivered straight to your inbox, subscribe to the Substack. Drop a comment with which part of the prefix sum trick you want explained differently, and share this with anyone prepping for coding interviews.

Quick Review

Two non-overlapping subarrays with given sums — what's the tell?

Fixed-target subarray sum + need positions/lengths → prefix sums + hashmap of sum→earliest end index.

Time complexity for this two-pass prefix-sum solution?

$O(n)$ — one pass builds prefix sums, one pass combines best-left with best-right per split point.

Space complexity of the prefix-sum + best-length tracking approach?

$O(n)$ — prefix sum array plus a running best-length array for combining left/right splits.

Common bug when combining two subarrays here?

Only pairing subarrays found on opposite sides of a split; forgetting a valid pair can both lie fully left of some index.

This pattern vs plain 'subarray sum equals K'?

Same prefix-sum trick, but you also track minimum length ending by each index to later combine two disjoint valid windows.

Interview follow-up: extend to three non-overlapping subarrays?

Track best pair-sum-of-lengths ending by index (from this 2-subarray solution), then combine with a third window scanning left-to-right.

Find Two Non-overlapping Sub-arrays Each With Target Sum — free Python cheat sheet